

Steel Surface Crack Detection using Transfer Learning and Explainable AI

Course: AI Project EST

Project Objective: Implementation of an automated system for steel plate inspection using Convolutional Neural Networks, specifically targeting micro-level surface defects.

1. Import Libraries

Utilization of TensorFlow for deep learning, Scikit-Learn for performance metrics, and Seaborn for statistical visualization.

```
In [1]: import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import cv2

# Check for hardware acceleration
print("Number of GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
```

Number of GPUs Available: 1

2. Dataset Loading and Preprocessing

The dataset is processed using rescaling and data augmentation techniques to enhance model generalization in industrial environments.

```
In [2]: IMG_SIZE = (224, 224)
BATCH_SIZE = 32
DATASET_PATH = 'dataset/'

datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    vertical_flip=True,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training',
    shuffle=True
)

val_data = datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation',
    shuffle=False
)

print("Class Indices:", train_data.class_indices)
```

Found 1600 images belonging to 2 classes.

Found 400 images belonging to 2 classes.

Class Indices: {'crack': 0, 'no_crack': 1}

Found 400 images belonging to 2 classes.

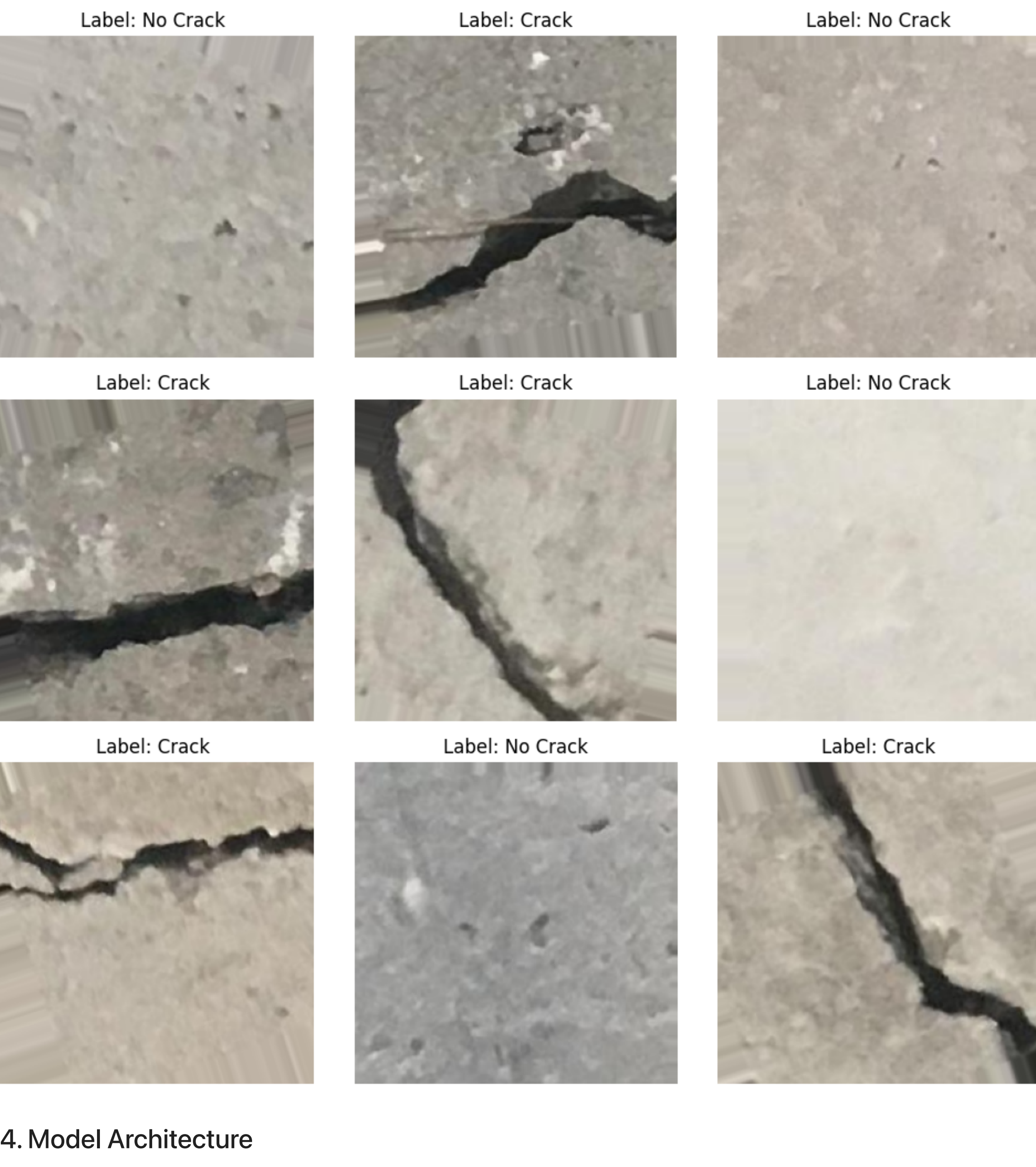
Class Indices: {'crack': 0, 'no_crack': 1}

3. Sample Visualization

Visual inspection of the dataset labels and image quality.

```
In [3]: def show_samples(data_gen):
    images, labels = next(data_gen)
    plt.figure(figsize=(10, 10))
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i])
        label = "Crack" if labels[i] == 0 else "No Crack"
        plt.title(f"Label: {label}")
        plt.axis("off")
    plt.tight_layout()
    plt.show()

show_samples(train_data)
```



4. Model Architecture

Implementation of Transfer Learning using a pre-trained MobileNetV2 backbone. A custom classification head is appended for the binary classification task.

```
In [4]: base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = False # Freeze base model for initial training

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Metal device set to: Apple M4

systemMemory: 24.00 GB

maxCacheSize: 8.88 GB

5. Training Methodology

Phase 1: Classifier Head Training

Training the custom dense layers while keeping the feature extraction layers frozen.

```
In [5]: print("Commencing Phase 1 training...")
history_phase1 = model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)

Commencing Phase 1 training...
Epoch 1/5
2026-05-05 23:07:11.826064: W tensorflow/tsl/platform/profile_utils/cpu_utils.cc:128] Failed to get CPU frequency: 0 Hz
50/50 [=====] - 12s 157ms/step - loss: 0.0666 - accuracy: 0.9744 - val_loss: 0.0200 - val_accuracy: 0.9900
Epoch 2/5
50/50 [=====] - 7s 136ms/step - loss: 0.0155 - accuracy: 0.9937 - val_loss: 0.0118 - val_accuracy: 0.9925
Epoch 3/5
50/50 [=====] - 7s 142ms/step - loss: 0.0102 - accuracy: 0.9956 - val_loss: 0.0110 - val_accuracy: 0.9925
Epoch 4/5
50/50 [=====] - 7s 146ms/step - loss: 0.0168 - accuracy: 0.9944 - val_loss: 0.0098 - val_accuracy: 0.9925
Epoch 5/5
50/50 [=====] - 7s 148ms/step - loss: 0.0104 - accuracy: 0.9969 - val_loss: 0.0214 - val_accuracy: 0.9925
```

Phase 2: Model Fine-Tuning

The base model is unfrozen and retrained with a lower learning rate to optimize the feature extraction layers for this specific domain.

```
In [6]: print("Commencing Phase 2 (Fine-tuning)...")
base_model.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_fine = model.fit(
    train_data,
    validation_data=val_data,
    epochs=3
)

WARNING:absl:At this time, the v2.11+ optimizer `tf.keras.optimizers.Adam` runs slowly on M1/M2 Macs, please use the legacy Keras optimizer instead, located at `tf.keras.optimizers.legacy.Adam`.
WARNING:absl:There is a known slowdown when using v2.11+ Keras optimizers on M1/M2 Macs. Falling back to the legacy Keras optimizer, i.e., `tf.kera
s.optimizers.legacy.Adam`.
Commencing Phase 2 (Fine-tuning)...
Epoch 1/3
50/50 [=====] - 21s 298ms/step - loss: 1.2199 - accuracy: 0.7044 - val_loss: 0.0161 - val_accuracy: 0.9925
Epoch 2/3
50/50 [=====] - 16s 315ms/step - loss: 0.1542 - accuracy: 0.9425 - val_loss: 0.0062 - val_accuracy: 1.0000
Epoch 3/3
50/50 [=====] - 16s 329ms/step - loss: 0.0973 - accuracy: 0.9631 - val_loss: 0.0146 - val_accuracy: 0.9975
```

6. Performance Evaluation

Quantitative analysis using Classification Reports and Confusion Matrices.

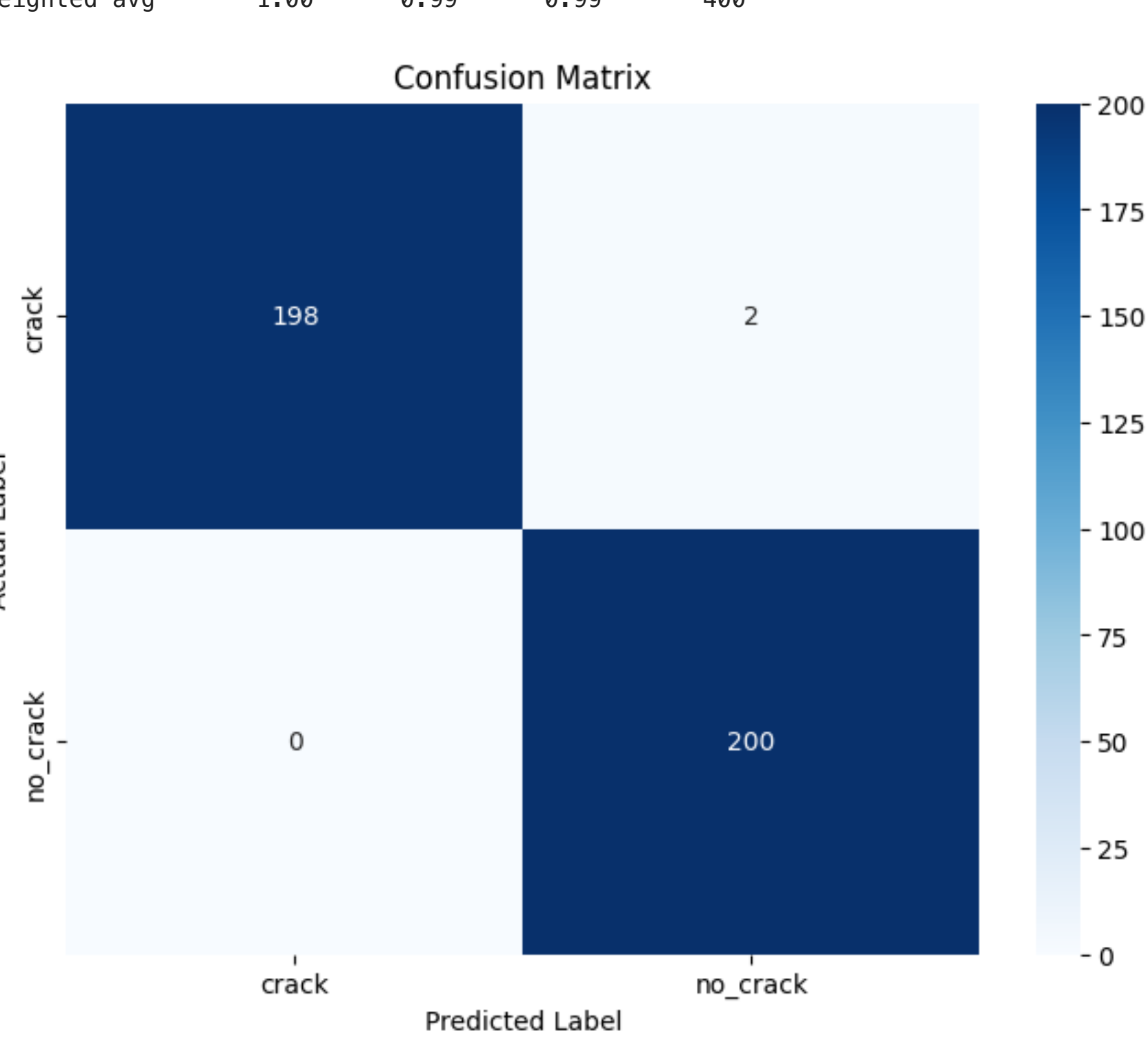
```
In [7]: val_data.reset()
y_true = val_data.classes
y_pred_prob = model.predict(val_data)
y_pred = (y_pred_prob > 0.5).astype(int).flatten()

print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=list(train_data.class_indices.keys())))

plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=train_data.class_indices.keys(),
            yticklabels=train_data.class_indices.keys())
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.title('Confusion Matrix')
plt.show()
```

13/13 [=====] - 2s 127ms/step

	precision	recall	f1-score	support
crack	1.00	0.99	0.99	200
no_crack	0.99	1.00	1.00	200
accuracy			0.99	400
macro avg	1.00	0.99	0.99	400
weighted avg	1.00	0.99	0.99	400



7. Explainable AI: Grad-CAM Visualization

Interpretability through Class Activation Maps (Grad-CAM) to identify feature importance in model decision making.

```
In [8]: def get_gradcam_heatmap(img_array, model, last_conv_layer_name):
    grad_model = tf.keras.models.Model([model.inputs], [model.get_layer(last_conv_layer_name).output, model.output])
    with tf.GradientTape() as tape:
        last_conv_layer_output, preds = grad_model(img_array)
        class_channel = preds[:, 0]
        grads = tape.gradient(class_channel, last_conv_layer_output)
        pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
        last_conv_layer_output = last_conv_layer_output[0]
        heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)
        heatmap = tf.math.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
        return heatmap.numpy()

def display_gradcam(img_path):
    img = tf.keras.utils.load_img(img_path, target_size=(224, 224))
    img_array = tf.keras.utils.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    heatmap = get_gradcam_heatmap(img_array, model, 'out_relu')
    heatmap = np.uint8(255 * heatmap)
    jet = cm.get_cmap("jet")
    jet_colors = jet(np.arange(256))[:, :3]
    jet_heatmap = jet_colors[heatmap]
    jet_heatmap = tf.keras.utils.array_to_img(jet_heatmap).resize((224, 224))
    jet_heatmap = tf.keras.utils.array_to_img(jet_heatmap)

    superimposed_img = jet_heatmap * 0.4 + (img_array[0] * 255)
    superimposed_img = tf.keras.utils.array_to_img(superimposed_img)

    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1); plt.imshow(superimposed_img); plt.title("Original Sample"); plt.axis("off")
    plt.subplot(1, 2, 2); plt.imshow(superimposed_img); plt.title("Grad-CAM Attribution Map"); plt.axis("off")
    plt.show()

try:
    sample_path = os.path.join(DATASET_PATH, 'crack', os.listdir(os.path.join(DATASET_PATH, 'crack'))[0])
    display_gradcam(sample_path)
except: pass
```

8. Predictive Analytics

Functionality for inference on novel image samples.

```
In [9]: def predict_image(path):
    img = tf.keras.utils.load_img(path, target_size=(224, 224))
    x = tf.keras.utils.img_to_array(img) / 255.0
    x = np.expand_dims(x, axis=0)
    pred = model.predict(x)[0][0]
    label = "No Crack" if pred > 0.5 else "Crack"
    confidence = pred if pred > 0.5 else 1 - pred
    print(f"Prediction Result: {label} (Confidence: {confidence*100:.2f}%)")
    plt.imshow(img); plt.axis('off'); plt.show()
```

9. Conclusion

The implemented system demonstrates a robust pipeline for automated surface inspection. Future developments will focus on integrating more diverse defect categories and optimizing for real-time inference latency.